

Linked List in Data Structures

Contents

1	Introduction	2
2	What is Linked List in Data Structures?	2
3	Example of Linked List	2
4	Structure or Representation of Linked List	2
5	Types of Linked List in Data Structures	3
5.1	Singly Linked List	3
5.2	Doubly Linked List	3
5.3	Circular Linked List	4
6	Uses of Linked Lists	4
7	Linked List Operations	5
7.1	Insertion	5
7.2	Deletion	6
7.3	Search	6
7.4	Traversal	7
7.5	Update	7
8	Advantages of Linked Lists	7
9	Disadvantages of Linked Lists (Limitations)	7
10	Array vs Linked List Difference	8
11	Implement Linked List in Java	8
12	Time Complexity of Linked List Operations	11
13	Implement Linked List in Python	11
14	Implement Linked List in C Programming	12
15	Implement Linked List in C++	14

1 Introduction

A linked list is a flexible data structure that consists of elements called nodes, each containing data and a reference to the next node. Unlike **arrays**, linked list data structures do not store their elements in contiguous memory locations; instead, each node points to the next, forming a chain.

This setup allows for efficient insertions and deletions, as you can add or remove nodes without rearranging the entire structure.

For example, imagine a train where each coach is a node linked to the next. You can easily add or remove a coach without affecting the others. Here, we'll explain linked list in data structure in detail with examples.

2 What is Linked List in Data Structures?

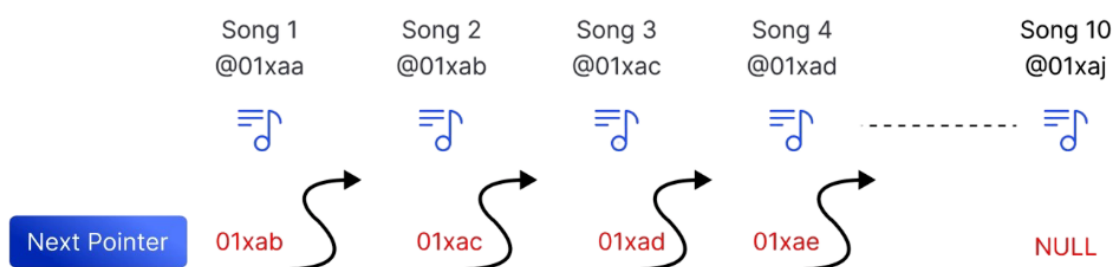
A linked list in data structures is a collection of elements called nodes, where each node contains some data and a reference to the next node in the sequence. This setup allows elements to be easily added or removed without reorganizing the entire structure.

Linked lists are crucial in **data structures and algorithms (DSA)** because they provide a dynamic way to manage data, adapting the size of the structure as needed without the high overhead associated with arrays.

3 Example of Linked List

Consider a real-life application of linked list data structure like a music playlist.

Each song in the playlist can be thought of as a node in a linked list, with each node pointing to the next song. Adding or removing songs from your playlist doesn't require shifting all other songs; you simply adjust the links.



4 Structure or Representation of Linked List

- **Node:** Each element of a linked list is called node. A node has two components:

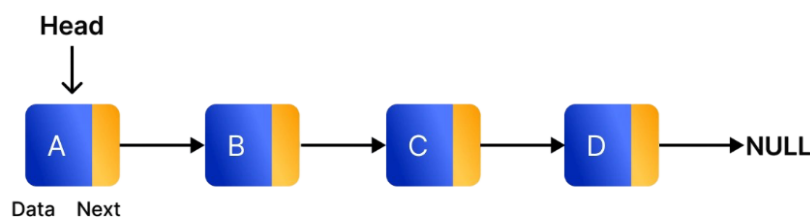
1. **Data:** The actual value or information that needs to be stored.
 2. **Next:** A pointer (or reference) to the next node in the linked list.
- **Head:** The first node in a linked list is called the head. It is the entry point to the list and used as a reference point to traverse it.
 - **Null:** The last node of a linked list, which points to null, indicating the end of the list.

5 Types of Linked List in Data Structures

There are three primary types of linked lists:

5.1 Singly Linked List

A **singly linked list** is a collection of nodes where each node contains data and a reference to the next node in the sequence. This structure allows for a simple, one-directional traversal from the head to the end of the list.



Example:

Consider managing a simple task list. Each task points to the next one until the end:

Task1 -> Task2 -> Task3 -> null

Operations like adding or removing tasks involve adjusting the 'next' reference of the preceding node, which is straightforward but only allows moving forward through the list.

5.2 Doubly Linked List

Doubly linked lists expand on singly linked lists by having each node contain two references: one pointing to the next node and one to the previous node. This allows nodes to be traversed in both forward and backward directions.



Example:

Imagine a playlist where you can move to the next or previous song:

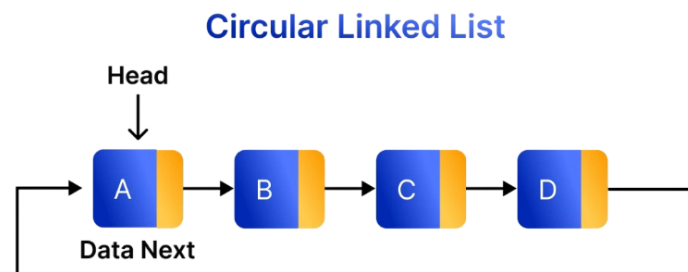
```
null <- PrevSong <-> CurrentSong <-> NextSong -> null
```

This setup is particularly useful for applications like image viewers or document viewers where users may need to go forward and backward between images or pages.

5.3 Circular Linked List

A circular linked list is a type of data structure where each node points to the next node, and the last node points back to the first, forming a circle.

This setup allows for an endless cycle of node traversal, which can be particularly useful in applications where the end of the list naturally reconnects to the beginning.

**Example:**

In multiplayer games, a circular linked list manages player turns, cycling back to the first player after the last one finishes.

6 Uses of Linked Lists

Data structure linked lists offer several practical uses across different applications:

1. Dynamic Memory Allocation

Linked lists are ideal for applications where the amount of data is unknown in advance and can change dynamically. They are extensively used in implementing memory management **algorithms**.

2. Implementing Stacks and Queues

Linked lists provide a natural way to implement other abstract data types like **stacks** or **queues**.

For example, a singly linked list can serve as a stack and queue by adding or removing nodes from one end.

3. Browser History Management

The functionality of back and forward navigation in web browsers can be effectively managed using a doubly linked list where each node points to the previous and next web pages visited.

4. Undo Functionality in Applications

Linked lists are used in implementing undo mechanisms in software applications, where operations are stored in a list and can be reversed by traversing backwards.

5. Music Playlists and Image Viewers

Applications that require sequential access to elements, such as media players or image viewers, often use doubly linked lists to facilitate easy navigation between items (next and previous).

6. Graphs Representation

Adjacency lists, which are used to represent graphs, can be implemented using linked lists, where each node represents a vertex and its linked list represents the adjacent vertices.

7. Polynomial Arithmetic

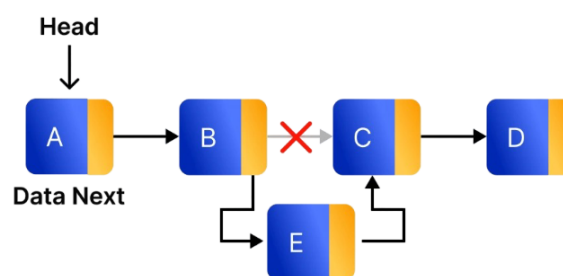
Linked lists are useful in representing and manipulating polynomials. Each node can represent a term of the polynomial, allowing for dynamic adjustments to the polynomial as terms are added or subtracted.

7 Linked List Operations

Understanding how to perform operations on linked lists is crucial for using them effectively in programming.

7.1 Insertion

Insertion in a linked list includes adding a new node to the list. You can insert a node at the beginning, in the middle, or at the end of the list.



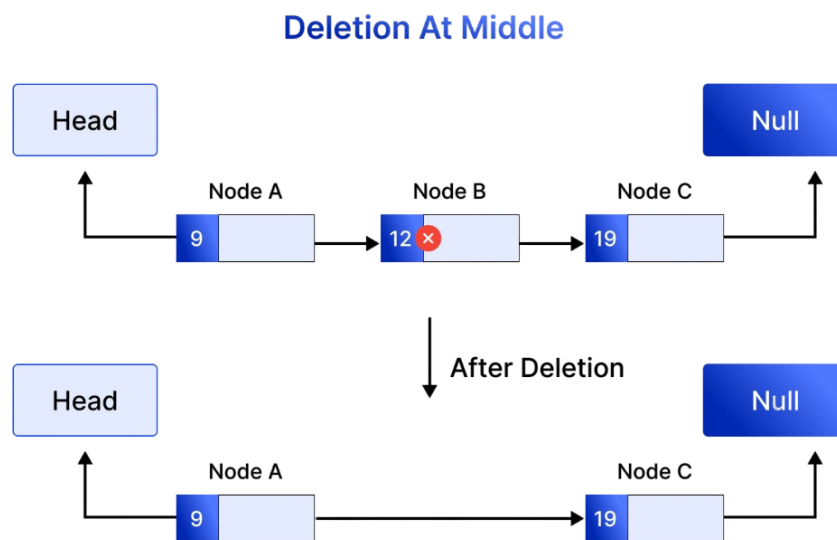
Example:

Inserting at the beginning: Imagine you have a list of daily tasks, and you suddenly need to add a new task at the top of your list. You create a new node for this task and make it the new head of the list, pointing to the previous first task.

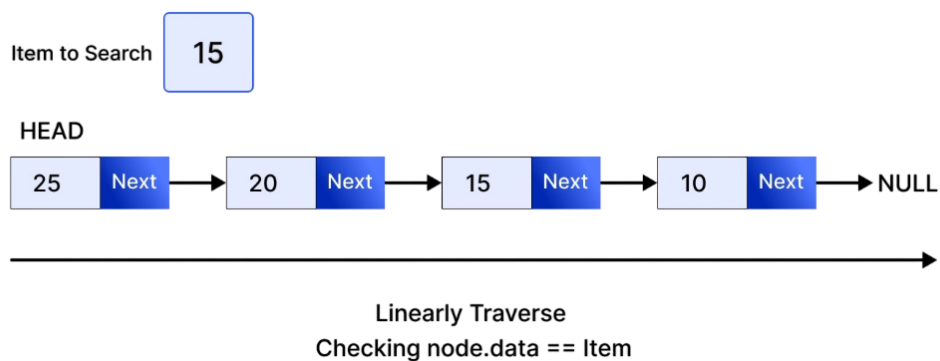
"Read book" -> becomes new head -> "Go to market" -> "Write essay" -> null

7.2 Deletion

Deletion removes a node from the linked list. This can also be done at the beginning, at a specific position, or at the end.

**7.3 Search**

Searching in a linked list involves traversing through the list from the head to find a node containing a specific value.



7.4 Traversal

Traversal means going through each node in the list from the beginning to the end to access or modify data.

7.5 Update

Updating involves changing the data in one of the nodes without altering the structure of the list.

8 Advantages of Linked Lists

- **Dynamic Sizing:** Linked lists can grow and shrink during runtime as they do not have a fixed size, unlike arrays.
- **Efficient Insertions and Deletions:** Adding or removing nodes does not require the elements to be contiguous in memory, so operations can be performed without reallocating or reorganizing the entire structure.
- **No Memory Waste:** Since there's no need to pre-allocate memory, linked lists can manage data more efficiently without wasting memory on unused space.
- **Ease of Implementation:** Can easily be implemented and manipulated, particularly when it comes to complex data structures such as stacks, queues, and other abstract data types.
- **Flexible Structure:** Nodes can easily be rearranged by changing their next pointers without the need for data movement, which is beneficial for applications that require frequent reordering of data.

9 Disadvantages of Linked Lists (Limitations)

- **Higher Memory Use:** Each node in a linked list requires additional memory for a pointer to the next (and possibly previous) node, which is more memory-intensive than the tight data packing in arrays.
- **Slower Access Time:** Direct access is not feasible with linked lists. To access an element at a specific index, you have to traverse the list from the head to that position, which takes more time than array access.
- **Complexity in Navigation:** Navigating a linked list can be more complex compared to navigating an array, as it requires pointer manipulation, which can be prone to errors like pointer corruption or memory leaks.
- **Extra Memory for Pointers:** The requirement for storing pointers typically means using extra memory, which can be significant in systems with a large number of elements.

10 Array vs Linked List Difference

Let's know about the difference between linked list and array in data structures:

Feature	Linked List	Array
Memory Allocation	Dynamic allocation. Nodes are allocated as needed.	Static or dynamic allocation but fixed size once created.
Element Access	Sequential access. Must traverse from the head.	Direct access via indices.
Insertion/Deletion	Fast operations as they only require pointer changes.	Slower operations, especially in the middle, because elements must be shifted.
Memory Efficiency	Less memory-efficient due to extra space for pointers.	More memory-efficient; only stores the data.
Implementation	More complex implementation, especially for beginners.	Simpler implementation and easier to manage.
Speed of Access	Slower to read due to traversal needs.	Fast and immediate access to any element.
Data Structure Size	Can grow or shrink dynamically without a high cost.	Requires resizing and copying to grow, which is costly.
Cache Locality	Poor cache locality due to non-contiguous storage.	Better cache locality due to contiguous memory layout.
Usage Scenarios	Good for scenarios with frequent insertions and deletions without a predetermined size of data.	Best when the size is known in advance and fast element access is needed.
Manipulation Overhead	Low overhead for adding/removing elements.	Higher overhead for adding/removing elements due to shifting.
Structural Overhead	Nodes carry extra overhead due to storing pointers.	No extra overhead beyond the data itself.

- Linked Lists are preferable when you need a flexible data structure that frequently changes size and where insertion and deletion speed is critical.
- Arrays are suitable for situations requiring fast access to elements, where the size of the data structure can be known beforehand, and operations like sorting and random access are common.

11 Implement Linked List in Java

Below is an implementation of a singly linked list in Java. This example includes methods to insert nodes at the beginning and end of the list, to search for elements, and to print all elements in the list, providing a comprehensive introduction to managing linked lists in Java.



```
1 public class Node {  
    public int data;  
    public Node next;  
  
    public Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```



```
1 public class LinkedList {  
    Node head;  
  
    public LinkedList() {  
        this.head = null;  
    }  
  
    // Method to insert a node at the beginning of the linked list  
    public void insertAtBeginning(int data) {  
        Node newNode = new Node(data);  
        newNode.next = head;  
        head = newNode;  
    }  
  
    // Method to insert a node at the end of the linked list  
    public void insertAtEnd(int data) {  
        Node newNode = new Node(data);  
        if (head == null) {  
            head = newNode;  
        } else {  
            Node last = head;  
            while (last.next != null) {  
                last = last.next;  
            }  
            last.next = newNode;  
        }  
    }  
  
    // Method to search for an element in the linked list  
    public boolean search(int key) {  
        Node current = head;  
        while (current != null) {  
            if (current.data == key) {  
                return true;  
            }  
            current = current.next;  
        }  
        return false;  
    }  
}
```



```

33     while (current != null) {
34         if (current.data == key) {
35             return true;
36         }
37         current = current.next;
38     }
39     return false;
40 }
41 // Method to print the linked list
42 public void printList() {
43     Node current = head;
44     while (current != null) {
45         System.out.print(current.data + " -> ");
46         current = current.next;
47     }
48     System.out.println("NULL");
49 }
50
51 public static void main(String[] args) {
52     LinkedList list = new LinkedList();
53
54     list.insertAtBeginning(10); // List: 10 -> NULL
55     list.insertAtBeginning(20); // List: 20 -> 10 -> NULL
56     list.insertAtEnd(30);       // List: 20 -> 10 -> 30 -> NULL
57     list.insertAtEnd(40);       // List: 20 -> 10 -> 30 -> 40 -> NULL
58
59     list.printList(); // Output: 20 -> 10 -> 30 -> 40 -> NULL
60
61     if (list.search(30)) {
62         System.out.println("Element found in the list.");
63     } else {
64         System.out.println("Element not found in the list.");
65     }
66 }
67 }

```

Explanation:

- *insertAtBeginning()*: This method creates a new node and inserts it at the front of the list, updating the head to the new node.
- *insertAtEnd()*: This method adds a new node to the end of the list. It traverses the list to find the last node and attaches the new node to it.
- *search()*: This method traverses the list from the head to search for a node containing the specified key.
- *printList()*: This method prints all the elements in the list from the head to the last node, followed by "NULL" to indicate the end of the list.

12 Time Complexity of Linked List Operations

Operation	Worst Case Complexity	Average Case Complexity
Access	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$
Insertion		
- At the beginning	$O(1)$	$O(1)$
- At the end	$O(n)$	$O(n)$
- In the middle	$O(n)$	$O(n)$
Deletion		
- At the beginning	$O(1)$	$O(1)$
- At the end	$O(n)$	$O(n)$
- In the middle	$O(n)$	$O(n)$

13 Implement Linked List in Python

Below is the linked list implementation in Python that includes methods to handle basic operations like appending new nodes, inserting nodes at the beginning, and displaying the list content.



```

1 class Node:
    # Creating a node
3     def __init__(self, item):
        self.item = item
5         self.next = None

7 class LinkedList:
    def __init__(self):
9         self.head = None

11     def append(self, item):
        """Append item to the end of the list."""
13         new_node = Node(item)
        if self.head is None:
15             self.head = new_node
            return
17         last = self.head
        while last.next:
19             last = last.next
        last.next = new_node

21     def insert_at_beginning(self, item):
23         """Insert an item at the beginning of the list."""
        new_node = Node(item)

```



```

25     new_node.next = self.head
      self.head = new_node

27
28     def print_list(self):
29         """Print all items in the list."""
30         current = self.head
31         while current:
32             print(current.item, end=" ")
33             current = current.next
34         print() # For new line at the end of the print.
35
36     if __name__ == '__main__':
37         linked_list = LinkedList()

38
39         # Append items
40         linked_list.append(1)
41         linked_list.append(2)
42         linked_list.append(3)
43
44         # Insert at the beginning
45         linked_list.insert_at_beginning(0)

46
47         # Print the linked list item
48         linked_list.print_list() # Output should be: 0 1 2 3

```

Explanation:

- **Node Class:** Defines a node with two attributes: item which holds the data and next which points to the next node in the list.
- **__init__:** Initializes a new linked list with a head that starts as None.
- **append:** Adds a new node with the specified item to the end of the list. It checks if the list is empty; if it is, the new node becomes the head. Otherwise, it traverses to the last node and links the new node to it.
- **insert_at_beginning:** Adds a new node at the beginning of the list. It sets the new node's next to the current head and then updates the head to the new node.
- **print_list:** Traverses the list from the head and prints each item. Continues until it reaches the end of the list (None).

14 Implement Linked List in C Programming

Below is the linked list implementation in C language that shows how to create a linked list, insert nodes at the beginning, at the end, and traverse through the list to print the values of the nodes.



```

#include <stdio.h>
2 #include <stdlib.h>

4 // Definition of the node
typedef struct Node {
6     int data;
    struct Node *next;
8 } Node;

10 // Function to create a new node
Node *createNode(int data) {
12     Node *newNode = (Node *) malloc(sizeof(Node));
    if (!newNode) {
14         printf("Memory error\n");
        return NULL;
16     }
    newNode->data = data;
18     newNode->next = NULL;
    return newNode;
20 }

22 // Insert a node at the beginning of the linked list
void insertAtBeginning(Node **head, int data) {
24     Node *newNode = createNode(data);
    newNode->next = *head;
26     *head = newNode;
}

28 // Insert a node at the end of the linked list
30 void insertAtEnd(Node **head, int data) {
    Node *newNode = createNode(data);
32     if (*head == NULL) {
        *head = newNode;
34         return;
    }
36     Node *last = *head;
    while (last->next != NULL) {
38         last = last->next;
    }
40     last->next = newNode;
}

42 // Function to print the linked list
44 void printList(Node *node) {
    while (node != NULL) {
46         printf("%d -> ", node->data);
        node = node->next;
48     }
    printf("NULL\n");
50 }

```



```

52 // Main function to demonstrate the linked list operations
   int main() {
54     Node *head = NULL; // Start with the empty list

56     // Inserting nodes
       insertAtBeginning(&head, 10); // List: 10->NULL
58       insertAtBeginning(&head, 20); // List: 20->10->NULL
       insertAtEnd(&head, 30); // List: 20->10->30->NULL
60       insertAtEnd(&head, 40); // List: 20->10->30->40->NULL

62     // Printing the linked list
       printf("Created Linked List: ");
64       printList(head);

66     return 0;
   }

```

Explanation:

- **Node Structure:** Defines a Node structure with an integer data field and a pointer next to the next node.
- **createNode Function:** Allocates memory for a new node, initializes it with data, and returns the new node.
- **insertAtBeginning Function:** Inserts a new node at the beginning of the linked list. It updates the head of the list to this new node.
- **insertAtEnd Function:** Inserts a new node at the end of the linked list. It traverses the list to find the last node and attaches the new node to it.
- **printList Function:** Traverses the linked list from the head and prints each node's data followed by '->'. It prints 'NULL' at the end of the list.

15 Implement Linked List in C++

Implementing a linked list in C++ allows for a more structured and object-oriented approach compared to C. The following is a detailed example of how to implement a singly linked list in C++ with methods to insert nodes at the beginning and end of the list, and to print all elements of the list.



```

1 #include <iostream>

3 using namespace std;

5 // Node class representing a single node in the linked list

```



```

class Node
7 {
  public:
9   int data;
   Node *next;
11
  public:
13   Node(int data) :
       data(data),
15   next(nullptr) {} // Constructor to initialize a node
};

17 // LinkedList class to handle operations on the linked list
19 class LinkedList
  {
21  public:
       Node *head;
23
  public:
25   LinkedList() :
       head(nullptr) {} // Constructor to initialize the linked list
27
   // Function to insert a node at the beginning of the linked list
29   void insertAtBeginning(int data) {
       Node *newNode = new Node(data);
31   newNode->next = head;
       head = newNode;
33   }

35   // Function to insert a node at the end of the linked list
   void insertAtEnd(int data) {
37   Node *newNode = new Node(data);
       if (head == nullptr) {
39   head = newNode;
       } else {
41   Node *temp = head;
       while (temp->next != nullptr) {
43   temp = temp->next;
       }
45   temp->next = newNode;
       }
47   }

49   // Function to print the linked list
   void printList() const {
51   Node *temp = head;
       while (temp != nullptr) {
53   cout << temp->data << " -> ";
       temp = temp->next;
55   }
       cout << "NULL\n";
57   }

```



```

59 // Destructor to clean up the dynamically allocated nodes
~LinkedList() {
61     Node *current = head;
        while (current != nullptr) {
63         Node *next = current->next;
            delete current;
65         current = next;
        }
67     head = nullptr;
    }
69 };

71 int main() {
    LinkedList list;
73
    // Inserting nodes
75     list.insertAtBeginning(10); // List becomes: 10 -> NULL
        list.insertAtBeginning(20); // List becomes: 20 -> 10 -> NULL
77     list.insertAtEnd(30); // List becomes: 20 -> 10 -> 30 -> NULL
        list.insertAtEnd(40); // List becomes: 20 -> 10 -> 30 -> 40 -> NULL
79
    // Printing the linked list
81     cout << "Created Linked List: ";
        list.printList();
83
    return 0;
85 }

```

Explanation:

- **Node Class:** Defines a single node of the linked list. Each node contains an integer data and a pointer next to the next node.
- **Constructor:** Initializes the head of the linked list to nullptr.
- **insertAtBeginning:** Creates a new node and inserts it at the beginning of the list. Updates the head to the new node.
- **insertAtEnd:** Creates a new node and appends it at the end of the list. This function traverses the list to find the last node and attaches the new node.
- **printList:** Traverses the linked list from the head and prints each node's data followed by '->'. Ends with 'NULL'.
- **Destructor:** Ensures that all dynamically allocated nodes are properly deleted to prevent memory leaks.